

SAÉ 201 Développement d'une application

Sujet : Find My Word

PATHMARANJAN Kuruparan
DOS SANTOS Nathanaël

BUT Informatique Venus

Enseignante responsable du groupe : Mme Azzag

S O M M A I R E

1. Avant-propos	2
2. Introduction et Compréhension du Sujet	3
3. Modélisation (simplifiée)	3
4. Analyse de la Modélisation et Modification (simplifiée)	6
4.1. Suppression de l'héritage entre Word et WordRepository	6
4.2. Implémentation du Polymorphisme obligatoire de la SAE	6
4.3. Modélisation (simplifié) après modification	7
5. Bilan	10

1. Avant-propos

Ce rapport présente la conception, la modélisation et l'implémentation du projet **Find My Word**, réalisé dans le cadre de la SAE 201.

Afin d'alléger ce document et d'éviter les répétitions, les informations techniques détaillées concernant la structure du projet, l'organisation des fichiers, les commandes de compilation, les commandes d'exécution ainsi que les différentes remarques d'utilisation ont été regroupées dans le fichier **README.md** fourni avec le projet.

Nous invitons donc le lecteur à consulter attentivement ce fichier avant toute tentative de compilation ou d'exécution du programme. Il contient notamment :

- l'arborescence complète du projet
- les commandes nécessaires à la compilation des sources Java
- les commandes permettant de lancer l'application
- les explications relatives aux options de compilation (-d) et d'exécution (-cp)
- les contraintes concernant le répertoire d'exécution du projet
- les règles de fonctionnement et d'utilisation du jeu.

2. Introduction et Compréhension du Sujet

La SAE, Find My Word, nous demande de concevoir une version simplifiée du jeu Wordle.

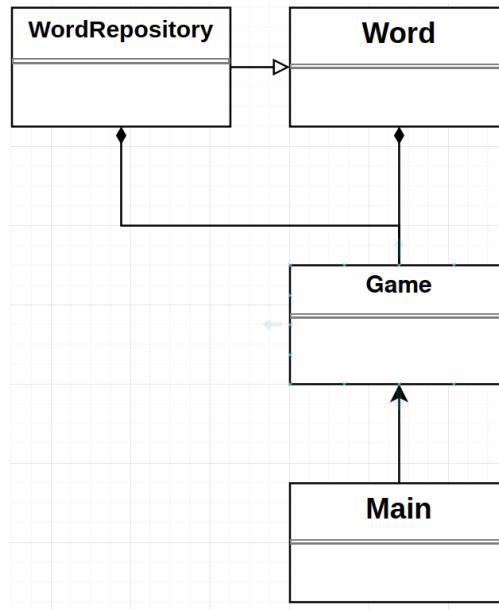
Le déroulé du jeu est le suivant : un joueur dispose de 6 tentatives pour deviner un mot secret de 5 lettres.

Le sujet de la SAE impose des critères stricts comme : respect des principes de la programmation orientée objet (encapsulation, séparation des responsabilités), propreté du code et utilisation obligatoire du polymorphisme.

3. Modélisation (simplifiée)

D'après ce qu'on a compris sur sujet de la SAE, la modélisation simplifiée doit comporter les classes suivantes :

- Game (gère la partie),
- Word (représentant un mot du jeu) ,
- WordRepository (permettant de récupérer un mot parmi une liste de mots),
- Main (lancer le programme)



- Main **utilise** Game
- Game a un lien **d'agrégation** avec Word, WordRepository
- WordRepository a un lien **d'héritage** avec Word

Nous avons considéré suffisant pour un diagramme UML simplifié

Après avoir analysé le fonctionnement du jeu, les attributs et opérations (méthodes) sont pour chaque classe (ce qu'on a identifié pour l'instant) :

Main :

- aucun attribut, juste une méthode main, cette classe a comme rôle de seulement lancer le programme

Game :

- Attribut Nom pour le nom du joueur
- Attribut Tentatives pour les tentatives du joueur
- Attribut WordGame pour le mot secret (avec la classe Word)
- Attribut WordRepository pour une liste de mots
- Accesseur getNom
- Mutateur setNom
- Accesseur getTentatives
- Mutateur setTentatives
- Accesseur getMotS
- Mutateur setMotS
- Accesseur getWordRepo
- Mutateurs setWordRepo
- Méthode run qui lance la partie
- Méthode choose qui permet de choisir un mot (paramètre)
- Méthode choose qui permet de choisir un mot (sans paramètre)

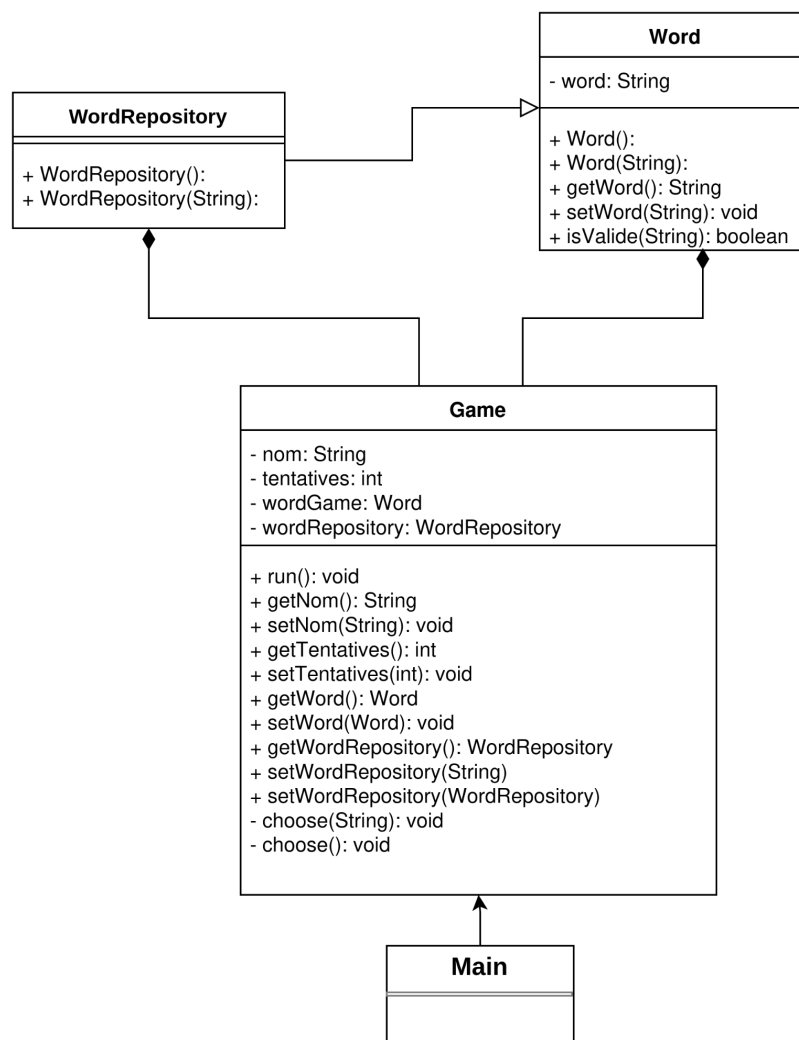
Word :

- Attribut Word qui est le mot
- Constructeur Word qui prend un mot parmi word.json
- Constructeur Word qui prend en paramètre un mot
- Accesseur getWord
- Mutateur setWord
- Méthode isValid qui permet de vérifier la validité d'un mot

WordRepository :

- Constructeur WordRepository qui ne prend rien en paramètre et qui récupère un mot parmi une liste de mots (aléatoire)
- Constructeur WordRepository qui prend en paramètre une chaîne de caractère (mot) et qui le cherchera parmi une liste de mots (pour les tests)
- (3 autres méthodes lui sont hérité de Word)

Voici le diagramme après les précédents ajouts :



4. Analyse de la Modélisation et Modification (simplifiée)

4.1. Suppression de l'héritage entre Word et WordRepository

Dans la première analyse, nous avons pensé qu'il y aurait un lien d'héritage entre WordRepository et Word. Après relecture du cours, cette approche a été abandonnée. En effet, l'héritage traduit une relation "est un" (ex: Chien est un Animal). Or, un dépôt de mots (WordRepository) n'est pas un mot (Word) mais une "collection" de mots. À la place, nous avons mis en place une relation d'association : WordRepository manipule/donne des objets Word. Donc :

- WordRepository a un lien **d'association** avec Word

4.2. Implémentation du Polymorphisme obligatoire de la SAE

Le cahier des charges de la SAE impose l'utilisation du polymorphisme (pour, par exemple, changer de "mode de jeu" avec un mot aléatoire via un fichier JSON et les tests automatisés avec mot fixe imposé).

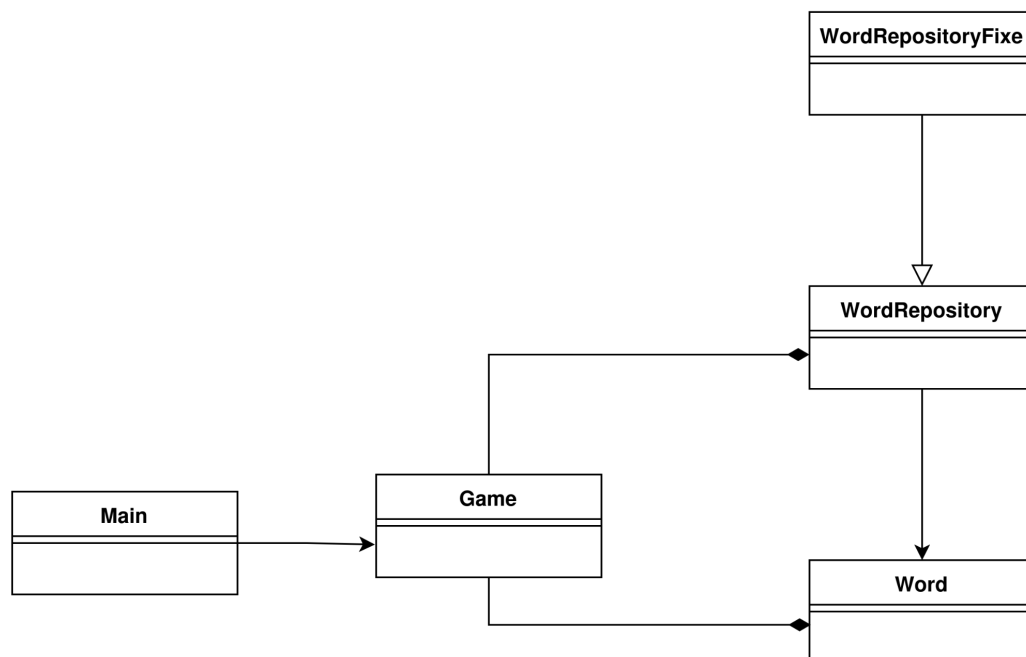
Pour répondre à cette contrainte, nous mettons le polymorphisme par **héritage de deux nouvelles classes** :

- WordRepository (Classe Mère) : Méthode générale de récupération d'un mot avec comme méthode getWord().
- WordRepositoryFixe (Classe Fille) : Elle hérite de WordRepository et est précisément créée pour les phases de test. Elle possède un attribut pour stocker un mot fixe et redéfinit la méthode getWord() pour renvoyer toujours ce même mot (d'où le polymorphisme)

Dans la classe Game, l'attribut reste de type WordRepository. Donc :

- WordRepositoryFixed a un lien **d'héritage** avec WordRepository

4.3. Modélisation (simplifié) après modification



- **Main a un lien d'association avec Game**
- **Game a un lien de composition avec Word et WordRepository**
- **WordRepository a un lien d'association avec Word**
- **WordRepositoryFixe a un lien d'héritage avec WordRepository**

Après avoir analysé le fonctionnement du jeu et vérifié que tout était bon, les attributs et opérations (méthodes) sont pour chaque classe (ce qu'on a identifié après sont en **bleu**) :

(certains éléments ont été ajoutés pour améliorer la compréhension)

Main : Aucun attribut, juste une méthode main, cette classe a comme rôle de seulement lancer le programme.

Game :

- **Attributs** :
 - nom (String) pour le nom du joueur
 - tentatives (int) : pour le nombre d'essais (maximum 6)
 - WordGame (Word): pour le mot secret
 - WordRepository (WordRepository): pour la collection des mots
- **Méthodes** :
 - Accesseur getNom / Mutateur setNom.
 - Accesseur getTentatives / Mutateur setTentatives.
 - Accesseur getMotSecret / Mutateur setMotSecret.

- **Constructeur Game(WordRepository repo) : permet de mettre le dépôt de mots, qu'il soit réel ou de test, pour le polymorphisme).**
- Méthode run() : qui lance la partie.

Word :

- **Attributs :**
 - Word (String) : qui est le mot
- **Méthodes :**
 - Constructeur Word qui prend en paramètre un mot.
 - Accesseur getWord / Mutateur setWord
 - isValid qui permet de vérifier la validité d'un mot
 - **analyse() : qui prend en paramètre un Word pour la comparaison du mot secret au mot proposé et génère le tableau (String[]) d'étiquettes OK/PRESENT/ABSENT de la SAE).**

WordRepository

- **Attributs :**
 - **wordsList (List<String>) : permet de stocker la liste globale des mots issus du fichier JSON**
- **Méthodes :**
 - Constructeur WordRepository() qui ne prend rien en paramètre et qui charge le fichier JSON.
 - **Méthode getWord() : méthode principale qui choisit et retourne un mot de manière aléatoire dans la liste pour le jeu réel.**

WordRepositoryFixe

- **Attributs :**
 - **fixeWord (String) : stocke le mot unique imposé pour les test**
- **Méthodes :**
 - **Constructeur WordRepositoryFixe() : qui prend en paramètre un String et met en place la collection avec le mot spécifique à tester.**
 - **getWord() : redéfinition de la méthode de la classe mère pour renvoyer juste le mot fixe au lieu d'un mot aléatoire (pour respecter le polymorphisme)**

Ceux Supprimé :

Game :

- Méthodes Accesseur getWordRepo / Mutateurs setWordRepo supprimé car Game n'a pas besoin de modifier sa collection de mots en cours de partie, il lui est fourni au départ.
- Méthode choose avec et sans paramètre : supprimé car c'est le rôle du WordRepository de fournir le mot. Game demande juste un mot à la collection.

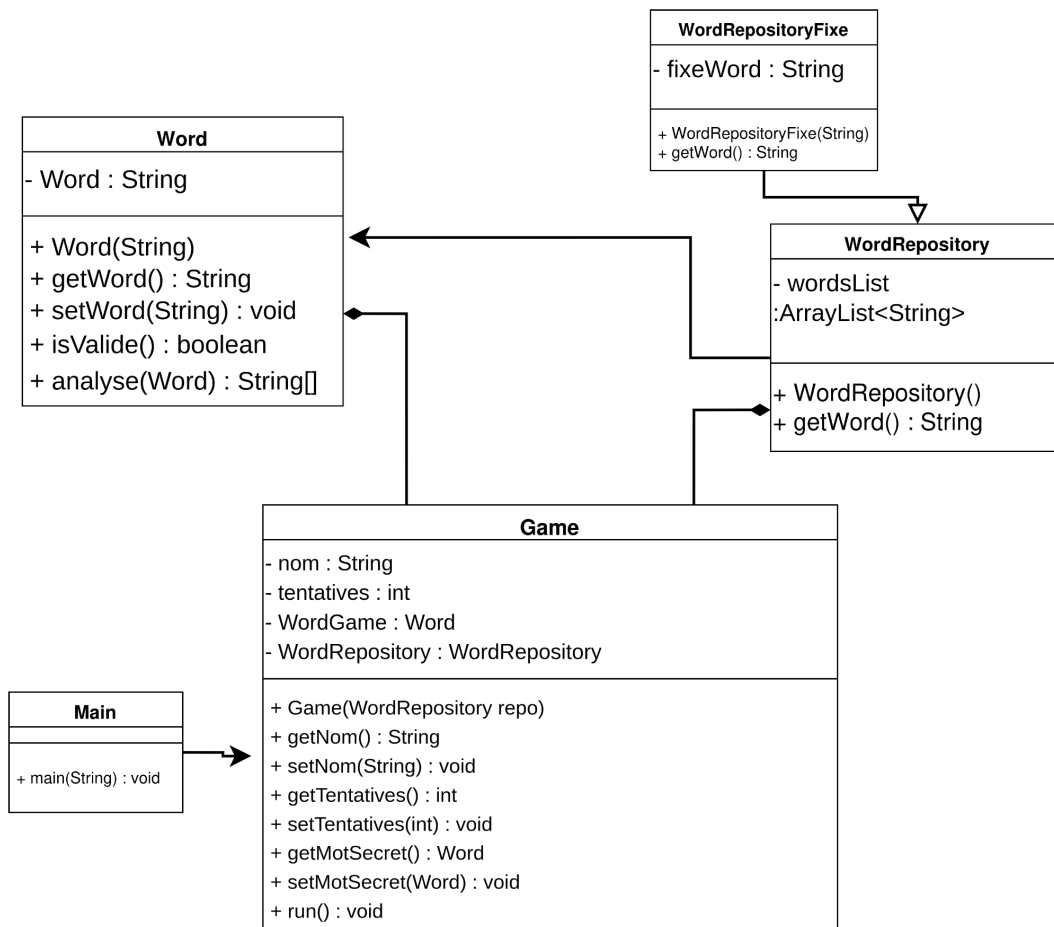
Word :

- Constructeur Word qui prend un mot parmi word.json supprimé car la classe Word représente un mot unique, elle n'a pas à savoir l'existence du fichier JSON, ce rôle appartient au WordRepository.

WordRepository :

- Constructeur WordRepository(String test) : qui prend en paramètre une chaîne de caractère (pour les tests), supprimé car cette logique a été transférée chez sa fille (WordRepositoryFixe) afin de respecter le fameux polymorphisme
- "(3 autres méthodes lui sont héritées de Word)" supprimé dû à la suppression de l'ancien héritage "WordRepository → Word" (WordRepository n'hérite plus d'aucune méthode de Word).

Voici le diagramme après les précédents ajouts et les modifications apportées :



Bilan de l'implémentation :

Ce qui a été ajouté ou modifié :

- La classe WordRepository (mère) est devenue une classe abstraite.

En effet, c'était mieux, pour la cohérence des demandes de la SAE et de ce que nous avons appris par rapport aux classes abstraites (cours 9), que la classe WordRepository devenait une classe abstraite pour accueillir 2 nouvelles classes héritées d'elle : WordRepositoryFixe (pas changé) et WordRepositoryAlea

- Game a maintenant une méthode d'affichage, de vérification de victoire et de gestion de temps et également deux nouveaux variables d'instance début et fin

En effet, nous avons remarqué que la méthode run() était "trop lourde" et avait une très grande responsabilité sur le lancement du jeu, l'affichage des grilles et les conditions de victoire/défaite. De ce fait, pour alléger cette méthode, les méthodes affichage(String[]), WIN(String[]), commencer() et stop() ont été ajoutées et également les variables début et fin (pour le temps)

- La classe WordRepository a désormais une méthode chargerJson()

En effet, dans la même logique que pour alléger la méthode run() dans Game, on a déplacé la responsabilité de charger du Json du constructeur de WordRepository vers une méthode.

- Suppression de l'attribut nom de Game et ajout de la classe Joueur. La classe Game a maintenant trois variable d'instance en plus: tour (int), tours (String[])

Ces ajouts sont là pour satisfaire une contrainte optionnelle du sujet : *"il gère les tours entre deux joueurs et cumule les points et les temps pour chaque joueur ; le nombre de tours est décidé parmi 1, 5, 10, 20"*. Cet ajout nous a énormément aidé par rapport à l'ajout de l'historique. De plus, le choix de supprimer "nom" dans Game est logique : ce choix supprime toute redondance ("nom" dans Game et dans Joueur)

- Ajout de la méthode afficherHistorique dans la classe Game

Cet ajout est là pour satisfaire une contrainte majeure du sujet : *"L'affichage du jeu (grille, messages, interactions utilisateur) doit être, autant que possible, séparé de la logique métier. Autrement dit : la classe Game gère la logique du jeu, l'affichage doit rester simple et ne pas mélanger calcul et présentation"*. Cet ajout nous aide énormément par rapport à la structure et à la propreté de notre code.

5. Bilan

Ce que nous avons fait est que nous avons fait une implémentation complète de la logique des mots via la classe Word. Dans cette dernière, il y a une gestion autonome des critères de validité (mots de 5 lettres, exclusivement alphabétiques et sans aucun doublon) ainsi que de l'algorithme d'analyse comparative des lettres (OK, PRESENT, ABSENT). Nous avons également modifié la classe WordRepository pour la rendre abstraite WordRepository, ainsi ses filles WordRepositoryAlea (mode de jeu réel) et WordRepositoryFixe (mode test) ont respectivement le rôle de donner un mot aléatoire et fixe.

De plus, dans Game, il y a une boucle globale qui gère la distribution des rôles (joueur 1 et joueur 2), la saisie des utilisateurs avec une boucle sécurisée (forçage si le mot est invalide) et les grilles textuelles de chaque essai.

On a également créé une classe Joueur encapsulant les indicateurs individuels, c'est-à-dire le nom, le temps accumulé et les points. Cela permet une gestion complète et simultanée de deux joueurs sur une même partie de jeu.

De plus, nous avons intégré un système d'historique en fin de session (en utilisant un tableau à deux dimensions) et aussi un système pour calculer l'écart de point et s'il y a une égalité, un départage au chronomètre pour désigner le vainqueur.

Ce qui n'a pas été fait est que malgré les modifications apportées sur UML, nous avons remarqué après coup que notre diagramme des classes finale révèle plusieurs importantes lacunes de conception graphique. En effet, d'une part, les cardinalités n'ont pas été mis sur les associations. D'autre part, une erreur s'est glissée dans la représentation des dépendances : le diagramme affiche un lien d'agrégation direct entre Game et la classe WordRepository (devenue abstraite). Or, lorsqu'on voit la classe, Game interagit avec les filles (sous-classes) concrètes de WordRepository (WordRepositoryFixe et WordRepositoryAlea). Le diagramme UML cible n'est donc pas pleinement fidèle à la réalité de notre code source.

Ce qui fonctionne est le filtrage des propositions via la méthode isValide() est fonctionnel (les saisies incorrectes ou contenant des doublons sont immédiatement signalées et l'utilisateur est contraint de ressaisir son mot sans perte de tentatives). Également, l'ajustement du calcul des scores (7 - tentatives) fonctionne à la perfection. Un joueur trouvant la solution au premier essai décroche bien le maximum (6 points, au lieu de 5), et un échec logique n'enlève aucun point (0 point).

Le programme alterne sans aucun problème entre les deux joueurs, calcule le temps écoulé par manche et met à jour en temps réel les variables cumulées au sein de chaque instance de Joueur.

L'historique restitue la structure de la partie. La méthode pour désigner le vainqueur fonctionne dans tous les cas de figure : victoire aux points, départage à la seconde près en cas d'égalité de score, ou déclaration d'égalité parfaite.

Le système pour rejouer réinitialise correctement les scores d'une session à l'autre lorsque l'utilisateur tape O.

Ce qui ne fonctionne pas est qu'à l'heure actuelle, le code source ne présente aucun bug fonctionnel bloquant. Toutes les boucles de contrôle s'exécutent à la perfection. La seule limite qu'on pourrait citer est la gestion des exceptions (comme une entrée de type texte là où le programme attend le nombre de tours de type entier via

Integer.parseInt()), qui peut provoquer un plantage de la JVM si l'utilisateur saisit délibérément une valeur "malveillante". Étant donné que le module Java qu'on a n'inclut pas les Exceptions, nous avons décidé de négliger les exceptions. Afin d'augmenter notre efficacité, nous avons choisi une approche mixte, combinant autonomie et programmation en binôme.

En binôme, on s'est chargées de la conceptualisation des classes, afin de poser les bases pour l'implémentation. Lorsqu'on est arrivé dans la partie implémentation, on s'est concentré sur la création des classes et des "méthodes simples" de certaines classes. Les phases de test, de repérage des erreurs (bugs) comme notamment l'accumulation des mots d'un tour à l'autre et de relecture du code, on également aussi été réalisées ensemble.

En autonomie, Nathanaël s'est concentré sur la mise en place du déroulement de jeu avec la méthode run() de Game, la gestion et l'intégration des sous-classes de WordRepository et de la classe Joueur. Kuruparan s'est lui concentré sur la mise en place du temps, la gestion du score et l'élaboration de l'historique (avec un peu d'aide de Nathanaël).

Au niveau des difficultés rencontrées, on a eu principalement un problème sur l'intégration de l'historique dans la méthode run(). Au départ, les propositions de mots de chaque manche s'accumulaient de manière désordonnée d'un tour à l'autre. Nous avons dû restructurer la portée de nos variables locales en comprenant que le tableau de suivi des essais devait être obligatoirement être réinstanciés à l'intérieur de la boucle principale des manches pour repartir à neuf à chaque changement de joueur.

On peut citer aussi le problème du système de points où la formule de notation des points pénalisait les victoires du premier coup (qui ne rapportait que 5 points au lieu de 6).

Nous avons également supprimer la redondance de l'attribut nom qui était initialement dans la classe Game qui est du coup dans la classe Joueur. Cela nous a évité de dupliquer inutilement des données en mémoire et a rendu notre code conforme aux principes de l'encapsulation de la POO. De plus, bien que le sujet évoquait la gestion de deux joueurs, nous avons choisi de ne pas stocker nos instances de joueurs dans un tableau d'objets figé au sein de Game. À la place, nous instancions nos objets de manière locale et indépendante dans la méthode run(). Les données accumulées sont donc directement extraites depuis chaque entité.

#Retour au début